

Sıfırdan yazmayalım. Depolamayı değiştirelim.

Çekirdek alan mantığı sağlam ve ölçümle kazanılmış bilgiyi taşıyor. Eksik olan yanlış kurulmuş değil, henüz hiç kurulmamış — ve bu ikisi çok farklı maliyetler.

27 Ağustos 2026 AnswerGap 6.591 satır kod Toplam harcama \$0,118

Karar

TAVSIYE

Mevcut mimarinin üzerinden gidelim. Yeniden yazmak, projenin en pahalı varlığını — ölçülerek öğrenilmiş alan bilgisini — çöpe atar ve karşılığında hiçbir şey kazandırmaz.

Bunu üç gerekçeyle söylüyorum, ve üçü de bugün doğruladığım şeylere dayanıyor.

Birincisi: çekirdek, kodun kendisinden daha değerli. `answergap/` altındaki 1.180 satır saf fonksiyon değil sadece — içine ölçülmüş kararlar gömülü. Dallanma çarpanının 8 değil 4 olduğu; click depth'in recursion'dan ~13 kat ucuz olduğu; Google'ın artık PAA'yı AI Overview ile cevapladığı ve kaynak URL kapsamının %0 olduğu; ve en kritiği, gap metriğinin "en yüksek örtüşme" değil *eşiği geçen sayfa sayısı* olduğu. Bu bulgular haftalarca ölçüm ve gerçek para karşılığı alındı. Yeniden yazmak kodu geri getirir, bu bilgiyi getirmez.

İkincisi: eksik olan yanlış değil, yok. Sökülecek kötü bir veritabanı şeması yok, yanlış seçilmiş bir ORM yok, dolaşmış bir kimlik doğrulama katmanı yok. Kalıcılık katmanı hiç bulunmayan bir prototip, doğru katmanı eklemek için mümkün olan en ucuz başlangıç noktasıdır. Yeniden yazma, kötü kurulmuş bir temeli olanlar için doğru cevaptır; bizimki henüz dökülmemiş.

Üçüncüsü: arayüz backend değişimine bağlı değil. Web katmanı API ile HTTP üzerinden, `lib/types.ts` 'te tanımlı tipli bir sözleşmeyle konuşuyor. Depolamayı tamamen değiştirsek bile o sözleşme korunduğu sürece arayüzün tek satırı değişmez. Bugün bunu fiilen test ettik: canlı crawl'ı ekledim, arayüz aynı tiplerle çalışmaya devam etti.

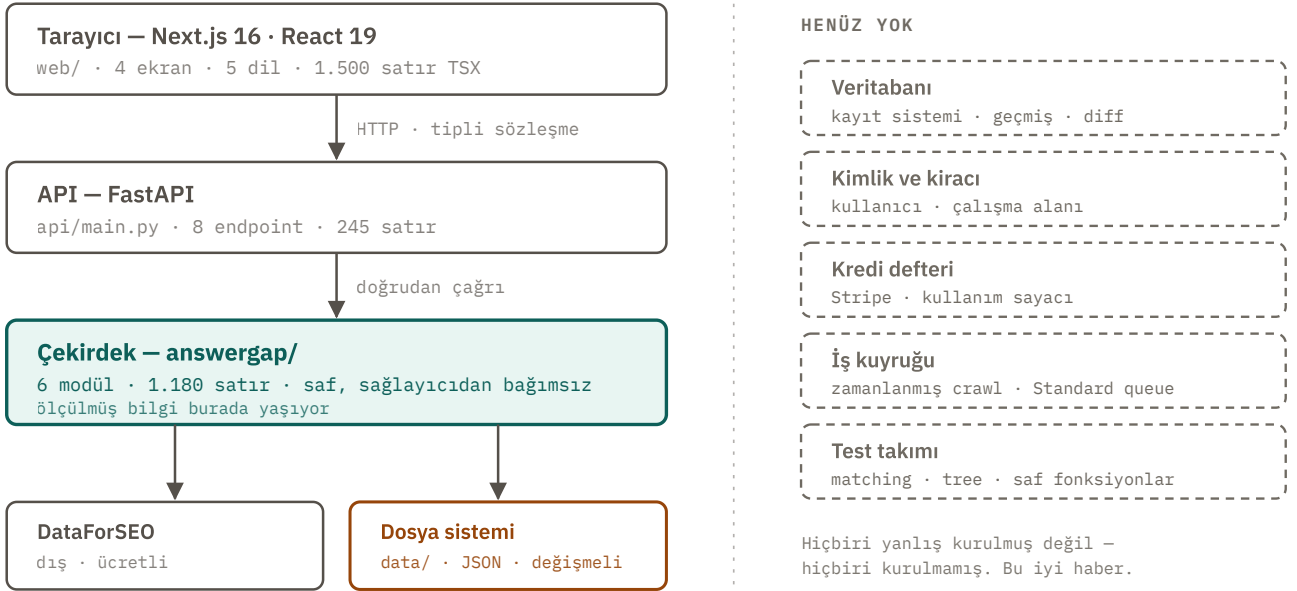
Ama dört şey değişmek zorunda

KONU	DURUM	NEDEN
Depolama	DEĞİŞMELİ	JSON dosyaları eşzamanlı yazma, TTL, sürüm geçmişi, kiracı ayrımı ve diff motorunu taşıyamaz. Bugün bu yüzden gerçek bir hata yaşadık.
Test	HIÇ YOK	Merkezî iddiası hesaplanmış bir metrik olan ve eşiği doğrulanmamış bir üründe sıfır test, en büyük tek risk.
İş çalıştırıcı	HIÇ YOK	Zamanlanmış crawl (ajans planı) ve Standard queue'ya geçiş, ikisi de asenkron iş gerektiriyor.
Kimlik · kiracı · kredi	HIÇ YOK	Kimse para ödeyemeden önce şart. Ama depolamadan <i>sonra</i> gelir — şeması ona oturur.

Sıra önemli: depolama diğer üçünün de üzerine oturduğu katman.

Şu anki mimari

Dört katman var ve üçü kurulu. Aşağıdaki şekil, "yeniden yazmayalım" argümanının neye dayandığını gösteriyor: solda duran zincir sağlam ve birbirine bağlı; sağdaki kutular ise bozuk değil, sadece *orada değil*.



Soldaki zincir uçtan uca çalışıyor ve bugün gerçek Google verisiyle doğrulandı. Sağdaki kesikli kutular yeniden yazma gerektirmez; çekirdeğe dokunmadan eklenirler. Tek istisna kehribar kutu: dosya sistemi yerinde duruyor ama değişmesi gerekiyor.

Klasör yapısı

Aşağıdaki ağaç diskteki gerçek durum. Renkler yukarıdaki semantiği sürdürüyor:

TEAL

kurulu ve doğrulanmış,

KEHRIBAR

var ama değişecek,

GRI

henüz yok.

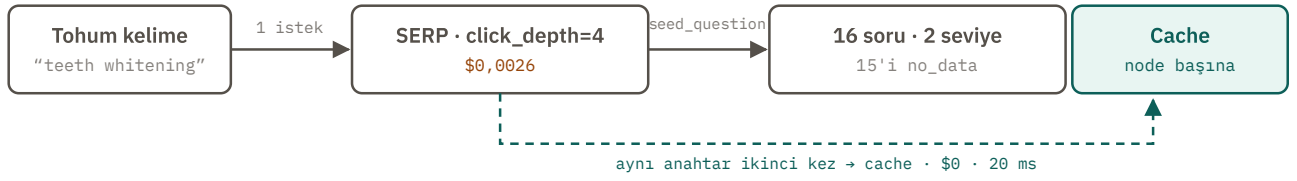
```
AnswerGap/
├─ answergap/                çekirdek alan mantığı – 1.180 satır, saf Python
│   ├─ dataforseo.py         346  API istemcisi: retry, bütçe tavanı, cache
│   ├─ languages.py          320  dil paketleri (en·tr·de·es·fr): stopword, eşanlam
│   ├─ live.py               498  canlı crawl + soru başına gap skoru [bugün]
│   ├─ matching.py           133  örtüşme skoru – ürünün merkezî hesabı
│   ├─ text.py               141  normalizasyon: dedup ve cache anahtarı
│   └─ tree.py               372  ağaç kurma, skortlama, durum sınıflandırma
├─ api/
│   └─ main.py               245  FastAPI · 8 endpoint · veritabanı yok
├─ web/
│   └─ app/                  Next.js 16 · React 19 · TypeScript
│       └─ page.tsx (arama) · tree/[slug] (ağaç ekranı)
│   └─ components/          QuestionTree · GapTable · QuestionDetail · Badge
│       └─ i18n/             en.ts tipi tanımlar; eksik anahtar build'i kırar
│       └─ lib/              api.ts (HTTP) · types.ts (sözleşme)
├─ data/                    ← mimari kararın düğümlendiği yer
│   ├─ raw/                 58 dosya · Faz 0 arşivi – KANIT, commit'li
│   ├─ live/serp/           canlı SERP cache · node başına · $0,0112
│   ├─ live/trees/          kurulu ağaçlar, tam JSON belge olarak ← hata kaynağı
│   ├─ locations/           213 ülke
│   └─ PHASE05_labels.csv   14 satır, 14'ü etiketsiz ← tıkanma noktası
├─ scripts/                 Faz 0/0.5 doğrulama scriptleri – 1.213 satır
├─ CLAUDE.md                ölçülmüş kuralların yazılı hali
├─ requirements.txt          pratikte boş – fastapi/uvicorn/pydantic eksik
├─ tests/                   yok
├─ migrations/              yok
└─ .github/workflows/       yok
```

Dikkat çekmek istediğim iki satır var. `data/raw/` bilerek commit'li: bunlar raporlardaki her sayının geri izlendiği kanıt, ve CLAUDE.md'nin deyimiyle tarihsel PAA verisi başka hiçbir yerde yok. `requirements.txt` ise pratikte boş – içindeki yorum "ürün kodu başladığında burası dolacak" diyor, o an geldi ama dolmadı. Şu an projeyi temiz bir makinede kuramazsın.

Bir aramanın yolculuğu

Para tam olarak nerede harcanıyor sorusunun cevabı, kredi modelinin de cevabı.
Bugün ölçtüğüm gerçek rakamlarla:

KEŞİF – 1 KREDİ



GAP SKORU – AYRI FİYATLI, SORU BAŞINA



Keşif ucuz, skortlama pahalı – ve bu ayrım kredi modelinin ta kendisi. 16 sorunun hepsini otomatik skortlamak aramayı ~15 kat pahalılaştırırdı; doğrulanmamış bir eşikle bunu yapmak ise parayı muhtemelen yanlış cevaba harcamak olurdu.

Veriler şu an nerede duruyor

Hiç konuşmadığımız konu buydu ve cevabı şu: **her şey düz JSON dosyalarında, diskte.** Veritabanı yok. API açılışta `data/raw/` 'u tarayıp ağaçları bellekte kuruyor; canlı aramalar `data/live/` 'a yazılıyor.

YER	İÇERİK	BOYUT	ROLÜ
data/raw/	58 dosya	17 MB	Faz 0 arşivi. Kanıt niteliğinde, commit'li. 15 MB'ı gitignore'lu konum dökümü.
data/live/serp/	5 yanıt	212 KB	Canlı SERP cache'i. Node başına anahtarlanmış — bu doğru.
data/live/trees/	2 ağaç	36 KB	Kurulmuş ağaçlar, tam JSON belge olarak. Sorun burada.
data/locations/	213 ülke	24 KB	Statik referans. Sorunsuz.

Ortalama bir SERP yanıtı 30 KB. 43 yanıt toplam 1,3 MB.

BUGÜN YAŞADIĞIMIZ GERÇEK HATA

CLAUDE.md açıkça şunu söylüyor: "Cache birimi node'dur, ağaç değil. Ağaç seviyesinde cache'lersen tek bayat node 70 çağrının tekrarlanmasına yol açar." SERP cache'ini node başına yazdım — doğru. Ama kurulmuş ağacı tek bir JSON belgesi olarak kaydettim, ve bu tam olarak kuralın yasakladığı şey.

Sonuç: bir soruyu skorladım (\$0,0020), sonra aynı aramayı tekrar çalıştırdım. Ağaç yeni PAA yanıtından yeniden kuruldu ve **parayla alınmış skoru sildi** — sayaç `covered: 2` 'den `covered: 1` 'e düştü. Belirtiyi yamayla kapattım (eski skorlar artık taşınıyor), ama sebep yama değil: *ağacı belge olarak saklamak*.

Bu, tek kullanıcı bir prototipte küçük bir aksaklık. Zamanlanmış crawl'lar ve eşzamanlı skorlama devreye girdiğinde aynı sebep sessiz veri kaybına döner.

Dosya sisteminin taşıyamayacağı dört şey

- Eşzamanlı yazma.** İki skorlama isteği aynı ağaç dosyasına yazarsa biri kaybolur. Kilit yok, atomik yazma yok.
- TTL ve stale-while-revalidate.** CLAUDE.md uyarlanabilir TTL istiyor (haber 6 saat, evergreen 90 gün, seviye 2+ iki katı). Dosya sisteminde son değişiklik tarihinden başka tutamak yok.
- Geçmiş ve diff.** "Tarihsel PAA verisi en savunulabilir varlığımız" diyoruz, ama üzerine yazan bir dosya geçmiş tutmaz. Diff motorunun karşılaştıracağı ikinci bir sürüm yok.

- **Kiracı ayrımı.** Ajans planı çoklu çalışma alanı vaat ediyor. Dosya yolunda kiracı kavramı yok.

Önerim: ağacı belge değil, satır olarak sakla

Tek cümlelik özet: **ağaç bir JSON belgesi değil, kenar satırlarından oluşan bir tablo olmalı.** Bu, bugünkü hatanın da doğrudan çözümü – yeniden crawl yeni satır ekler, mevcut skoru *silemez*, çünkü ikisi farklı satırlarda yaşar.

BUGÜN – TEK BELGE



Sonuç: soru B'nin skoru silinir.

Belge bir bütün olarak değiştirilir; içindeki ödenmiş satırı koruyacak bir sınır yoktur.

ÖNERİ – SATIRLAR



Sonuç: skor korunur, geçmiş birikir.

Diff motoru iki snapshot'ı karşılaştırır – ayrıca yazmaya gerek yok, veri zaten orada.

Aradaki tek yapısal fark, ağacın belge mi yoksa kenar satırları mı olduğu. Solda yeniden crawl bir yazma işlemidir ve her şeyin üstünden geçer; sağda bir ekleme işlemidir ve hiçbir şeyi bozamaz. Diff motoru ile "tarihsel PAA verisi bizim varlığımız" iddiası da sağdaki şemada bedavaya gelir.

Somut seçim: PostgreSQL + nesne depolama

Kayıt sistemi Postgres olsun. SQLite şaşırtıcı derecede uzağa götürür ama zamanlanmış crawl'lar ve eşzamanlı skrolama eşzamanlı yazar demektir; ayrıca diff motoru gerçek işlem ve indeks ister. Şu aşamada Neon veya Supabase ücretsiz katmanı fazlasıyla yeter – kendi sunucunu yönetme.

Ham SERP yanıtları satır değil, blob olsun. Ortalama 30 KB ve JSON ~8-10 kat sıkışıyor. Bunları Postgres'e koymak veritabanını şişirir ve hiçbir sorguya girmezler. Cloudflare R2 (çıkış ücreti yok) ya da benzeri bir S3 uyumlu depoya, aynı cache anahtarıyla, gzip'li yaz. Postgres'te sadece *çıkarılmış alanlar* dursun: başlık, URL, alan adı, örtüşme skoru.

Dosya sistemi cache'i yerel geliştirme için kalsın. Aynı arayüz, farklı arka uç. Bunu bugünden mümkün kılan şey, çekirdeğin zaten saf olması – `Client` bir `cache_dir` alıyor; onun yerine bir depo nesnesi almasını sağlamak küçük bir değişiklik.

Redis'i şimdi ekleme. İş kuyruğu ve hız sınırlama gerektiğinde gelir. Şu an eklemek, karşılığı olmayan bir işletme yükü.

Şema taslağı

TABLO	ANAHTAR ALANLAR	NEDEN BÖYLE
question	id · normalized_text · display_text · language_code · location_code	(normalized, lang, loc) üzerinde tekil. CLAUDE.md'nin cache anahtarının ta kendisi.
serp_snapshot	id · question_id · fetched_at · cost · blob_url · ttl_expires_at	Asla üzerine yazılmaz. Geçmiş burada birikir; TTL burada yaşar.
paa_edge	parent_question_id · child_question_id · snapshot_id · position	Ağaç budur. Belge değil kenar — tekrar sayısı da buradan bedava gelir.
gap_score	question_id · snapshot_id · threshold · strategy · matching_pages · status	Eşik ve strateji satırda saklanır: eşik değişince eski skorlar yalan olmaz.
crawl	id · workspace_id · seed_question_id · created_at	Kullanıcının bir araması. Geçmiş kısıtlanmaz (AlsoAsked'ın hatası).
label	question_id · verdict · user_id · created_at	Arayüzdeki başparmak. Eşik doğrulamasını kullanımın yan ürünü yapar.
workspace · user · credit_ledger	—	Kiracılık ve faturalama. Depolamadan sonra, en son gelir.

Kritik ayrıntı: **gap_score** eşiği ve stratejiyi kendi satırında taşır. Eşik henüz doğrulanmadığı için bu şart — değiştiğinde eski skorları yeniden yorumlayabilmek gerekir.

Bugüne kadar yapılanlar

AŞAMA	DURUM	ÇIKTI VE KANIT
Faz 0 doğrulama	BİTTİ	Dallanma çarpanı ölçüldü: 4 , 8 değil. Click depth recursion'dan ~13 kat ucuz çıktı. Şehir bazlı aramanın gereksiz olduğu iki kez ölçüldü (Jaccard 1,000). load_async_ai_overview'ın para yaktığı kanıtlandı.
Faz 0 bulgusu gap nasıl hesaplanır	BİTTİ	32/32 PAA elementi references: null döndü — kaynak URL kapsamı %0 . Gap, PAA bloğundan hesaplanamaz. Çözüm: soruyu ayrıca sorup organik sonuçları karşılaştırmak. Doğru metrik "en yüksek örtüşme" değil, eşiği geçen sayfa sayısı .
Prototip çekirdek + API + arayüz	BİTTİ	6 modüllük çekirdek, 8 endpoint'li API, 4 ekranlı Next.js arayüzü, 5 dil (eksik anahtar build'i kırıyor), 213 ülke, gerçek Google verisinden 3 demo ağaç.
Faz B canlı crawl	BUGÜN	Arama kutusu DataForSEO'ya bağlandı. seed_question alanının gerçek ebeveyni verdiği keşfedildi: tek istek, 16 node, iki gerçek seviye . Gap skoru soru başına ayrı adım. Cache isabetinde \$0 / 20 ms.
Eşik doğrulama Faz 0.5	TIKALI	PHASE05_labels.csv: 14 satır, 14'ü de etiketsiz . Eşik 0, 60 ve strateji synonyms hâlâ tahmin. Bilinen yanlış pozitif: "Can 60 year old teeth be whitened?" gap işaretleniyor ama "Can Senior Teeth be Whitened?" başlıklı sayfa onu cevaplıyor.

Toplam harcama \$0,118. Her sayı `data/raw/` altındaki bir yanıtla geri izlenebilir.

Sırada ne var

Bu sıra keyfi değil; her adım bir öncekinin açtığı kapıdan geçiyor. Numaralar gerçek bir bağımlılık zincirini gösteriyor.

01 Etiketleme — 20 dakika, \$0

Diskteki 14 soruyu G/N olarak etiketle, `phase05_evaluate.py` 'yi çalıştır.

Neden önce bu: asıl soru "eşik kaç olmalı" değil, "**sözlük yetiyor mu, yoksa embedding şart mı**". 14 etiket bunu cevaplamaya yeter ve cevabı mimariyi değiştirir. Embedding gerekiyorsa bunu depolama şemasını yazmadan *önce* bilmek gerekir — vektörlerin nerede duracağı şemayı etkiler.

02 Test takımı — çekirdek saf, kolay iş

`matching.py` ve `tree.py` saf fonksiyonlar; `data/raw/` altındaki gerçek yanıtlar hazır fıkstür.

Depolamayı değiştirmeden önce yapılmalı. Testi olmayan bir kodu taşımak, neyi bozduğunu bilmeden taşımaktır.

03 Depolama katmanı — Postgres + blob

Yukarıdaki şema. Dosya sistemi arka ucu yerel geliştirme için korunur.

Diğer her şeyin üzerine oturduğu katman. Diff motoru, TTL, geçmiş ve kiracılık bu adımdan sonra *eklenecek özellik* olmaktan çıkıp şemanın doğal sonucu olur.

04 Arayüzde geri bildirim

Skorlanmış her soruda "bu gerçekten gap mi?" başparmağı; `label` tablosuna yazar.

Etiketleme bir angarya olmaktan çıkıp kullanımın yan ürünü olur. Eşik sorunu böyle kalıcı olarak çözülür — 14 etiketle değil, binlerce ile.

05 İş kuyruğu ve Standard queue

Asenkron crawl, zamanlanmış taramalar, node seviyesinde hata toleransı.

Standard queue Live'dan ~3,3 kat ucuz. Şu an bilerek Live kullanıyoruz çünkü webhook bir dizüstüne ulaşamaz; sunucuya çıkınca bu sapma kapanır ve birim maliyet üçte bire iner.

06 Kimlik, kiracılık, kredi ve Stripe

Kullanıcı, çalışma alanı, kredi defteri, ödeme.

En sona bırakılıyor çünkü şeması depolama katmanına oturuyor ve gelirden önce doğrulanması gereken şey ürünün kendisi, faturalama değil.

SIRANIN DIŞINDA, HEMEN

`requirements.txt` doldurulmalı — `fastapi`, `uvicorn`, `pydantic` kurulu ama yazılı değil. Şu an proje temiz bir makinede kurulamıyor. Tek satırlık iş, ama kurulamayan bir proje devredilemez de.

Açık riskler

RISK	AĞIRLIK	DURUM
Eşik doğrulanmamış	YÜKSEK	Ürünün merkezî iddiası ölçülmemiş bir sayıya dayanıyor. Bilinen bir yanlış pozitif var. Arayüz bunu "Provisional threshold" rozetiyle söylüyor — dürüst, ama çözüm değil.
Eşanlam sorunu	YÜKSEK	60 year old ↔ senior gibi anlam eşleşmelerini kelime örtüşmesi göremiyor. Sözlük mü embedding mi — adım 01 cevaplayacak.
Sıfır test	YÜKSEK	Hesaplanmış bir metriği satan üründe hiç test yok. Adım 02.
PAA gün içi kararlılığı	BİLİNMIYOR	İki kararlılık ölçümü de arka arkaya yapıldı. Farklı saatlerde tekrarlanmadı.
Google PAA'yı değiştirir	KONTROL DIŞI	Zaten bir kez oldu: PAA cevapları AI Overview'a taşındı ve kaynak URL'leri kayboldu. Arşivlenen geçmiş, bu riske karşı elimizdeki tek sigorta.

AnswerGap · mimari kararı · 27 Ağustos 2026

Bu belgedeki her rakam bu tarihte diskteki dosyalardan ya da DataForSEO'nun kendi bildirdiği maliyetlerden okundu; hiçbirisi tahmin değil.

Kaynak: docs/mimari.html — yeniden üretilebilir.